# METROPOLIS VS. CLUSTER MONTE CARLO FOR 2D POTTS MODEL: SAMPLING EFFICIENCY ACROSS PHASE TRANSITION

YUHAN YE*

**Abstract.** We study single–spin Metropolis and single–cluster Wolff updates for phase transitions in two-dimensional lattice spin systems at zero external field. After a brief review of the Ising and Potts models, we introduce the Metropolis algorithm and its cluster counterparts—Swendsen–Wang and Wolff—emphasizing physical intuition, precise mathematical formulation, and basic convergence guarantees. A standard comparison inequality shows that Swendsen–Wang is not asymptotically worse than single-site heat-bath (Glauber) dynamics in regimes where the latter mixes rapidly.

Our numerical study focuses on implementation-independent measures of sampling efficiency rather than wall-clock time. Across temperature grids bracketing the transition, we first validate correctness by reproducing the internal-energy $u(T)$ and specific-heat $c(T)$ curves and locating the critical point. We then compare efficiency near $T_c$ using standard MCMC efficiency metrics (e.g., [10, 12]): integrated autocorrelation time (IACT), effective sample size (ESS), and a cost-normalized measure (Cost/ESS) that counts spin flips rather than CPU time. In these tests, Wolff typically achieves smoother thermodynamic curves and reduced IACT at comparable work.

**1. Introduction.** Metropolis–Hastings Markov chain Monte Carlo (MCMC) is a cornerstone algorithm for sampling from Gibbs measures and estimating ensemble averages in statistical physics. The algorithm originated with Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller (1953) as a method to compute the equation of state for a 2D system, motivated by the intractability of high-dimensional integrals in statistical mechanics. It was later generalized by Hastings (1970) to a broader statistical context, making it a "top ten" algorithm of the 20th century. Today, it is ubiquitous not only in physics but also in Bayesian statistics, machine learning, and computational finance for sampling from complex, high-dimensional probability distributions.

For lattice spin systems like Ising and Potts, naive single–spin M-H updates (the main competing algorithm in this context) experience *critical slowing down* near the phase transition. This severe increase in autocorrelation time motivated the development of cluster algorithms that decorrelate configurations more rapidly. Among these, the Swendsen–Wang (SW) algorithm and the Wolff single–cluster modification are widely used.

Many other variants of MCMC exist to tackle different problems; for instance, Hamiltonian Monte Carlo (HMC) uses physical dynamics to propose efficient moves in continuous spaces, while Parallel Tempering is designed to overcome complex, multi-modal energy landscapes. Authors widely regard the Metropolis algorithm as revolutionary for its simplicity and broad applicability, though its efficiency (i.e., mixing time) is known to be highly problem-dependent.

In this paper, we review the Ising/Potts models and the Metropolis and Cluster Update (Wolff) Algorithms, then present an implementation-agnostic comparison between the two updates on the 2D Potts model across its phase transition.

**2. Ising Models, Potts Model and Phase Transition.** The Ising model is one of the most fundamental models in statistical physics. It was proposed by Wilhelm Lenz in 1920 and first analyzed by his student Ernst Ising in 1925 (who solved the one–dimensional case), originally to explain the microscopic mechanism of ferromagnetic phase transitions in solids.

In modern computational/applied mathematics it appears as a binary Markov random field whose energy minimization connects to graph cuts, probabilistic inference, and combinatorial optimization, and as a canonical testbed for MCMC/sampling algorithms.

*18.335 Introduction to Numerical Methods, Massachusetts Institute of Technology, USA (yyh03@mit.edu).

The $q$-state Potts model extends Ising to multiple labels, enabling multi-class formulations used in clustering, image segmentation, and community detection, while retaining rich phase-transition behavior.

*Ising model..* We consider an $M = N \times N$ square lattice with periodic boundaries and nearest-neighbor interactions, and coupling $J > 0$. Let $\sigma = (\sigma_i)$ denote the spins on lattice sites $i \in \Lambda$. States are $\sigma_i \in \{-1, +1\}$ and the Hamiltonian is

$$H(\sigma) \;=\; -J\sum_{\langle i,j\rangle} \sigma_i\sigma_j \;-\; h\sum_i \sigma_i. \tag{2.1}$$

*q-state Potts model.* The Potts model is a direct generalization of the Ising model. It allows the spin at each lattice site to take one of $q$ possible states. States are $\sigma_i \in \{1, \ldots, q\}$ with Hamiltonian

$$H(\sigma) \;=\; -J\sum_{\langle i,j\rangle} \mathbf{1}\{\sigma_i = \sigma_j\} \;-\; h\sum_i \sigma_i. \tag{2.2}$$

Here, $\mathbf{1}\{\cdot\}$ is the indicator function, which is 1 if the spins are equal and 0 otherwise. When $q = 2$, the Potts model is equivalent to the Ising model.

*Gibbs measure.* At inverse temperature $\beta = (k_B T)^{-1}$ (we set $k_B = 1$ in the experiments), the Gibbs distribution is

$$\pi(\sigma) = \frac{1}{Z(\beta)}\, e^{-\beta H(\sigma)} \text{ , with partition function } Z(\beta) = \sum_\sigma e^{-\beta H(\sigma)}.$$

*Observables.* We define the ensemble (Gibbs) average of any observable $A$ by

$$\langle A(\sigma)\rangle_{\pi_\beta} \;:=\; \mathbb{E}_{\sigma\sim\pi_\beta}[A(\sigma)] \;=\; \sum_\sigma A(\sigma)\,\pi_\beta(\sigma). \tag{2.3}$$

The internal energy and its per–site version are

$$U(\beta) \;=\; \langle H(\sigma)\rangle_{\pi_\beta} \;=\; -\frac{\partial}{\partial\beta}\ln Z(\beta), \qquad u(\beta) \;=\; \frac{U(\beta)}{M}.$$

The specific heat and its per–site version are

$$C(T) \;=\; \frac{\partial U}{\partial T} \;=\; k_B\beta^2\Big(\mathbb{E}_{\pi_\beta}\big[H^2\big] - \mathbb{E}_{\pi_\beta}\big[H\big]^2\Big) \;=\; k_B\beta^2\,\frac{\partial^2}{\partial\beta^2}\ln Z(\beta), \qquad c(T) \;=\; \frac{C(T)}{M}.$$

It is often useful to track the magnetization $m(\sigma) = \frac{1}{M}\sum_{i\in\Lambda} s_i$ (with $s_i = \sigma_i$ for Ising, or $s_i = \mathbf{1}\{\sigma_i = r\} - 1/q$ for a fixed $r$ in Potts) and the susceptibility $\chi = M\beta\,\mathrm{Var}_{\pi_\beta}(m)$.

*Critical temperature and phase transition.* These systems exhibit a **phase transition**: as $T$ crosses a critical temperature $T_c$, macroscopic quantities, e.g., $u(T)$, $m(T)$, change sharply. Below $T_c$ the system exhibits long-range order (spins tend to align), while above $T_c$ it is disordered. Near $T_c$, the specific heat $c(T)$ exhibits either a divergence (for continuous transitions, $q \le 4$) or a finite latent-heat jump (for first-order transitions, $q > 4$); on finite lattices this typically appears as a sharp peak. For the 2D square-lattice Potts model,

$$\beta_c J \;=\; \ln\big(1+\sqrt{q}\big) \quad\Longleftrightarrow\quad T_c \;=\; \frac{J}{k_B\,\ln(1+\sqrt{q})}.$$

With $J = 1$ and $k_B = 1$ this reduces to $T_c = 1/\ln(1+\sqrt{q})$. A practical question arise:

*How can we estimate $T_c$ numerically on a computer?*

## 3. Metropolis Algorithm.

**3.1. Basic Idea with Physical Heuristic Interpretation.** The central difficulty of direct Monte Carlo (MC) sampling is that i.i.d. draws $\sigma^{(k)} \sim \pi_\beta(\sigma)$ are hard to obtain when

$$\pi_\beta(\sigma) \;=\; Z(\beta)^{-1} e^{-\beta H(\sigma)}, \qquad \beta = (k_B T)^{-1}.$$

To overcome this, Metropolis algorithm takes an "iteration" procedure to produce random variables, by constructing a Markov chain with transition kernel $P(\sigma \to \sigma')$ whose (under standard conditions) unique stationary distribution is the Gibbs measure $\pi_\beta(\sigma) \propto e^{-\beta H(\sigma)}$. Denoting the evolving law by $\nu_n$, one enforces $\pi_\beta P = \pi_\beta$ so that $\nu_{n+1} = \nu_n P$ contracts toward $\pi_\beta$ from arbitrary $\nu_0$ when the chain is ergodic, similar to the contraction mapping property as in fixed point iterations.

In statistical physics, macroscopic observables are ensemble averages over microscopic configurations. At equilibrium, one adopts the heuristic *ensemble average = time average*:

$$\langle H(\sigma) \rangle_{\pi_\beta} \;\approx\; \frac{1}{K} \sum_{k=1}^{K} H(\sigma^{(k)}), \tag{3.1}$$

provided the Markov chain explores the state space ergodically. The Metropolis construction supplies the "collision rule" (transition mechanism) that generates the sequence $\{\sigma^{(k)}\}$ consistent with the target Gibbs distribution.

**3.2. Mathematical Description.** An *equilibrium* (stationarity) requirement for a Markov kernel $P$ is

$$\pi_\beta P = \pi_\beta.$$

This condition alone does not determine $P$ (there are infinitely many such kernels), so one often adds the **detailed balance condition** (DBC, reversibility)

$$\pi_\beta(\sigma)\, P(\sigma \to \sigma') = \pi_\beta(\sigma')\, P(\sigma' \to \sigma),$$

which implies the odds

$$\frac{P(\sigma \to \sigma')}{P(\sigma' \to \sigma)} = \frac{\pi_\beta(\sigma')}{\pi_\beta(\sigma)} = e^{-\beta\, \Delta H}, \qquad \Delta H := H(\sigma') - H(\sigma).$$

A naive rule "accept if $\Delta H \le 0$, else accept with probability $e^{-\beta \Delta H}$" captures this ratio but by itself does not define a stochastic kernel (one still needs the probabilities of *proposing* moves). Hence we factor the transition into proposal and decision steps.

*Proposal–acceptance factorization.* For $\sigma' \neq \sigma$ set

$$P(\sigma \to \sigma') = Q(\sigma \to \sigma')\, A(\sigma \to \sigma'), \qquad P(\sigma \to \sigma) = 1 - \sum_{\tau \neq \sigma} P(\sigma \to \tau),$$

where $Q$ is the *proposal* probability and $A$ the *acceptance* probability. If $Q$ is symmetric, $Q(\sigma \to \sigma') = Q(\sigma' \to \sigma)$, then two standard choices that satisfy DBC are:

1. **Metropolis acceptance.**

$$A(\sigma \to \sigma') = \min\{1,\, e^{-\beta \Delta H}\}.$$

2. **Glauber acceptance.**

$$A(\sigma \to \sigma') = \left(1 + e^{\beta \Delta H}\right)^{-1}.$$

The proposal matrix $Q$ can also be unsymmetric, then DBC is realized in the decision step, which is called Metropolis-Hastings (MH) algorithm in statistical learning theory. In this case, the reversibility and $\pi$-invariance are enforced by the MH acceptance

$$A(\sigma \to \sigma') = \min\left\{1,\ \frac{\pi(\sigma')\, Q(\sigma' \to \sigma)}{\pi(\sigma)\, Q(\sigma \to \sigma')}\right\}.$$

*Example.* For the 2D Ising model on an $N \times N$ lattice ($M = N^2$ sites), two standard choices of $Q$ are:

(i) *Equi-probability* proposal: choose any $\sigma' \neq \sigma$ uniformly from the $N_t = 2^M$ configurations. That is, take $Q(\sigma \to \sigma') = 1/(N_t - 1)$, $\forall \sigma' \neq \sigma$; and $Q(\sigma \to \sigma) = 0$.

(ii) *Single flip* proposal (preferred in practice): pick a site uniformly and flip its spin. That is, take $Q(\sigma \to \sigma') = 1/M$ if $\sigma$ and $\sigma'$ differ at exactly one site, and 0 otherwise.

Now, we summarize the Metropolis update:

*Metropolis Algorithm Update.*

1. Generate the proposal state $\sigma'$ from the current state $\sigma_n$ according to some strategy.
2. Define $\Delta H = H(\sigma') - H(\sigma_n)$ and compute the acceptance

$$A(\sigma_n, \sigma') = \min\left\{1,\ \frac{\pi(\sigma')}{\pi(\sigma_n)}\right\} = \begin{cases} 1, & H(\sigma') \leq H(\sigma_n), \\ \exp(-\beta\, \Delta H), & \text{otherwise.} \end{cases}$$

3. Generate a random variable $r \sim \mathcal{U}[0, 1]$.
4. If $r \leq A(\sigma_n, \sigma')$, set $\sigma_{n+1} = \sigma'$; else set $\sigma_{n+1} = \sigma_n$. Return to *Step 1*.

Under DBC, $\pi$ is invariant. Together with ergodicity (e.g., irreducibility and aperiodicity on a finite state space; this is equivalent to *primitivity*), this ensures convergence to $\pi$ from any start; the statements below make this precise.

**3.3. Theory of Convergence.** The basic theoretical framework for Metropolis and other MCMC methods is the ergodic theory of time-homogeneous Markov chains; we collect two standard convergence results here. For rigorous proofs and further background, see [1].

Write the total-variation (i.e., $\ell_1$) norm for vectors as $\|v\| := \sum_i |v_i|$. A time-homogeneous Markov chain with transition matrix $P$ on a finite state space is called *primitive* if there exists $\tau \in \mathbb{N}$ such that $P^\tau > 0$ (all entries positive).

THEOREM 3.1 (Ergodic theorem). *If $P$ is primitive, there is a unique invariant distribution $\pi$, and for any initial distribution $\nu$,*

$$\lim_{n \to \infty} \|\nu P^n - \pi\| = 0.$$

THEOREM 3.2 (WLLN for time-homogeneous Markov chains). *Let $\{X_n\}$ be a realization of a primitive Markov chain with invariant distribution $\pi$. For any function $f$ with $\mathbb{E}_\pi[\,|f|\,] < \infty$, there exists a constant $C = C(f, P)$ such that for all $n \in \mathbb{N}$ and $\varepsilon > 0$,*

$$\Pr\left(\left|\frac{1}{n}\sum_{k=1}^{n} f(X_k) - \mathbb{E}_\pi[f]\right| > \varepsilon\right) \ \leq\ \frac{C}{n\,\varepsilon^2}.$$

*In conclusion,* $\frac{1}{n}\sum_{k=1}^{n} f(X_k) \xrightarrow{\Pr} \mathbb{E}_\pi[f]$.

That is, long-run sample averages along a single trajectory converge to the true $\pi$-expectations. Hence Monte Carlo estimates produced by Metropolis are consistent.

**4. Cluster Update Algorithm.** Single–site Metropolis updates suffer from *critical slowing down* near the critical temperature $T_c$. To avoid this, Swendsen and Wang [2] introduced a powerful *clustering* algorithm which, together with a modification by Wolff [3], almost completely eliminates critical slowing down. Indeed, one can explain the SW construction via data augmentation.

**4.1. Clusters View.** For the zero–field Ising model with $x_i \in \{-1, +1\}$,

$$\pi(x) \;\propto\; \exp\Big\{\beta J \sum_{\langle i,j\rangle} x_i x_j\Big\} \;\propto\; \prod_{\langle i,j\rangle} \exp\big\{\beta J(1 + x_i x_j)\big\},$$

since $1 + x_i x_j \in \{0, 2\}$. Introduce an auxiliary variable $u_{ij} \geq 0$ on each edge and define the joint density

$$\pi(x, u) \;\propto\; \prod_{\langle i,j\rangle} \mathbf{1}\big\{\, 0 \leq u_{ij} \leq \exp\big(\beta J(1 + x_i x_j)\big)\big\}\,.$$

Marginalizing $u$ yields the Ising Gibbs distribution. Conditionally,

$$u_{ij} \mid x \sim \mathrm{Unif}(0, \exp(\beta J(1 + x_i x_j)))\,, \qquad x \mid u: \quad u_{ij} > 1 \Rightarrow x_i = x_j \quad \text{(otherwise unconstrained)}.$$

Thus $u$ influences $x$ only through the *bond events* $b_{ij} = \mathbf{1}\{u_{ij} > 1\}$. Given $x$, for like–spin neighbors,

$$\mathbb{P}(b_{ij} = 1 \mid x) = 1 - e^{-2\beta J}, \qquad \mathbb{P}(b_{ij} = 1 \mid x) = 0 \text{ if } x_i \neq x_j,$$

so the open–edge connectivity is exactly the Fortuin–Kasteleyn (FK) clustering. The SW step is: place bonds between like–spin neighbors independently with the above probability; then, for each connected cluster, flip all spins with probability 1/2.

**4.2. Wolff's single–cluster modification.** Wolff's key change [3] is to grow and flip *one* cluster per iteration (rather than all clusters as in SW):

1. Choose a seed site $i$ uniformly, set $a \leftarrow x_i$, initialize $C \leftarrow \{i\}$.
2. Grow $C$ by flood–fill: for each boundary neighbor $j$ with $x_j = a$, add $j$ to $C$ independently with probability $p_{\text{add}} = 1 - e^{-2\beta J}$; iterate until no additions occur.
3. Flip the cluster: set $x_k \leftarrow -x_k$ for all $k \in C$.

Let $C$ be a grown $+1$ cluster. Let $m$ (resp. $n$) be the number of boundary edges from $C$ to exterior $+1$ (resp. $-1$) spins. Flipping $C$ changes the energy by $\Delta H = 2J(m - n)$, hence

$$\frac{\pi_{\text{new}}}{\pi_{\text{old}}} = e^{-\beta \Delta H} = e^{-2\beta J(m-n)} = e^{2\beta J(n-m)}.$$

For the proposal ratio, realizing $C$ requires *rejecting* each of the $m$ potential additions to exterior $+1$ sites, which occurs with probability $(1 - p_{\text{add}})^m = e^{-2\beta J m}$. In the reverse move (where $C$ is a $-1$ cluster), the analogous factor is $(1 - p_{\text{add}})^n = e^{-2\beta J n}$. Thus

$$\frac{Q_{\text{new}\to\text{old}}}{Q_{\text{old}\to\text{new}}} = \frac{(1 - p_{\text{add}})^n}{(1 - p_{\text{add}})^m} = \frac{e^{-2\beta J n}}{e^{-2\beta J m}} = e^{2\beta J(m-n)}.$$

Therefore,

$$A_{\text{old}\to\text{new}} = \min\Big\{1,\; \frac{Q_{\text{new}\to\text{old}}\;\pi_{\text{new}}}{Q_{\text{old}\to\text{new}}\;\pi_{\text{old}}}\Big\} = \min\{1, 1\} = 1.$$

This exact cancellation relies on spin–flip symmetry at $h = 0$, which ensures matching boundary statistics for forward and reverse growth. When $h \neq 0$, the symmetry is broken and additional devices (e.g., "ghost–spin" embeddings) are needed to retain acceptance = 1 or to design comparably efficient updates.

**4.3. Generalization to the Potts Model.** For the Ising model with spins $\sigma_i \in \{\pm 1\}$ and Hamiltonian $H_{\text{Ising}} = -J_{\text{Ising}} \sum_{\langle i,j \rangle} \sigma_i \sigma_j$, the FK/Wolff bond probability is $p_{\text{add}} = 1 - e^{-2\beta J_{\text{Ising}}}$. For the Potts form $H_{\text{Potts}} = -J_{\text{Potts}} \sum_{\langle i,j \rangle} \mathbf{1}\{\sigma_i = \sigma_j\}$, the bond probability is $p_{\text{add}} = 1 - e^{-\beta J_{\text{Potts}}}$. This shows that the Wolff single–cluster update extends directly to the Potts model. When $q = 2$, these parameterizations are equivalent because $\mathbf{1}\{\sigma_i = \sigma_j\} = \frac{1+\sigma_i\sigma_j}{2}$.

**4.4. Compare to Metropolis with Thoeretical Guarantee.** Now we compare the cluster algorithms (Swendsen–Wang and Wolff) with the single-site Metropolis/Glauber updates. Below we summarize the main asymptotic mixing-time results comparing Swendsen–Wang with the single-site heat-bath (Glauber) dynamics; see [7, 8, 9] for details and rigorous results.

*Total variation distance and mixing time.* For distributions $\mu, \nu$ on a finite state space $\Omega$, recall that the total-variation distance is $\|\mu - \nu\|_{\text{TV}} := \frac{1}{2} \sum_{x \in \Omega} |\mu(x) - \nu(x)|$.

For a Markov kernel $P$ with stationary law $\pi$, define the worst-case distance

$$d(t) \;:=\; \max_{x \in \Omega} \big\| P^t(x, \cdot) - \pi \big\|_{\text{TV}}, \qquad t_{\text{mix}}(\varepsilon) \;:=\; \min\{\, t \in \mathbb{N} : d(t) \leq \varepsilon \,\}.$$

*Spectral gap.* Assume $P$ is reversible with respect to $\pi$ and (for convenience) consider its lazy version $P_{\text{L}} = \frac{1}{2}(I + P)$ so all eigenvalues lie in $[0, 1]$. If $1 = \lambda_1 > \lambda_2 \geq \cdots \geq \lambda_{|\Omega|} \geq 0$ are the eigenvalues of $P_{\text{L}}$, the (discrete-time) spectral gap is

$$\gamma \;:=\; 1 - \lambda_2.$$

*Gap–mixing connection.* Let $\pi_{\min} := \min_{x \in \Omega} \pi(x)$. For any $\varepsilon \in (0, 1)$,

$$\frac{1}{2\gamma} \log \frac{1}{2\varepsilon} \;\leq\; t_{\text{mix}}(\varepsilon) \;\leq\; \frac{1}{\gamma} \Big( \log \frac{1}{\varepsilon} + \log \frac{1}{\pi_{\min}} \Big).$$

Hence a larger spectral gap implies faster mixing (smaller $t_{\text{mix}}$).

*Comparison inequality (SW vs. single-site Glauber).* Let $P_{\text{SW}}$ and $P_{\text{HB}}$ denote the Swendsen–Wang cluster kernel and the single-site Glauber (see Section 3.2) kernel for the same Potts/random-cluster model on a bounded-degree graph. A standard comparison yields a constant $c > 0$ (depending on $q$ and the maximal degree) such that

$$\gamma(P_{\text{SW}}) \;\geq\; c\, \gamma(P_{\text{HB}})\,.$$

By the bounds above,

$$t_{\text{mix}}^{\text{SW}}(\varepsilon) \;\lesssim\; \tfrac{1}{c}\, t_{\text{mix}}^{\text{HB}}(\varepsilon) \quad \text{(up to the } \log(1/\pi_{\min}) \text{ factor)},$$

Here we state the main theoretical conclusion of this paper:

**Cluster Update SW is not asymptotically worse than single-site heat-bath (Glauber) in regimes where heat-bath mixes rapidly(e.g., off-critical and, for Ising, also at $T_c$).**

*Remark on Wolff.* Wolff performs *single-cluster* flips rather than the global FK cluster update of SW, so the usual spectral-gap comparison tools (block dynamics / RC coupling) do not transfer directly. Consequently, there is no general theorem of the same form for Wolff; its advantage near criticality is primarily supported by empirical dynamic critical exponents rather than universal spectral-gap inequalities. In particular, at $h = 0$ the Wolff move has acceptance 1, so work is spent on *effective* state changes rather than rejected trials.

**5. Numerical Experiment Setup and Comparison Methodology.** We study the 2D $q$-state Potts model on an $N \times N$ square lattice (periodic boundary), with zero field ($h = 0$) and normalization $J = k_B = 1$. Spins are $\sigma_i \in \{1, \ldots, q\}$ and

$$H(\sigma) \;=\; -\sum_{\langle i,j \rangle} \mathbf{1}\{\sigma_i = \sigma_j\}, \qquad \pi_\beta(\sigma) \;=\; Z(\beta)^{-1} e^{-\beta H(\sigma)}, \quad \beta = T^{-1}.$$

Writing $M = N^2$, the internal energy and specific heat per site are

$$u(T) \;=\; \frac{1}{M}\,\mathbb{E}_{\pi_\beta}[H], \qquad c(T) \;=\; \frac{1}{M}\,\beta^2\,\mathrm{Var}_{\pi_\beta}(H),$$

which will be estimated from Monte Carlo output below.

**5.1. Overview of Experiments.** In the experiments, we apply *Metropolis* and *Wolff* updates to characterize the phase transition in the 2D Potts model. First, we *identify the critical temperature* by analyzing the empirical $u(T)$ and $c(T)$ curves over a temperature grid around $T_c$ and check consistency with the exact formula $T_c = 1/\ln(1+\sqrt{q})$. Second, we *compare critical-regime dynamics* by running each algorithm near $T_c$ under matched work units, reporting implementation-independent efficiency metrics. Finally, leveraging the higher efficiency of the Wolff update near criticality, we *probe spatial correlations* and extract the correlation length $\xi(T)$.

1. **Identify the critical temperature $T_c$ (validation test).** Construct $u(T)$ and $c(T)$ over a temperature grid bracketing $T_c$ and compare the estimated transition point with the known formula $T_c = 1/\ln(1+\sqrt{q})$. To accelerate equilibration within this *scan only*, we may use an *annealed warm-start* (using the previous equilibrium configuration as the next initializer when stepping in $T$). This convenience is not used in any dynamical comparison. The goal is to verify that both Metropolis and Wolff reproduce the correct $u(T)$ and $c(T)$ curves and locate $T_c$ consistently.
2. **Critical-regime dynamics (efficiency test and comparison).** At $T \approx T_c$, we start from fresh random states (*no warm-start*), discard a fixed burn-in, and record one energy sample per *sweep*. For Metropolis, one sweep $= N^2$ single-site attempts. For Wolff, we accumulate clusters until the *cumulative number of flipped spins* reaches $\geq N^2$; that batch counts as one sweep so that work units are matched across algorithms. We then compare statistical efficiency using implementation-independent metrics (below).

**5.2. Comparison Using Implementation-Independent Metrics.**

*Sweep as the time unit.* After discarding a burn-in prefix, we record one energy value per *sweep*. For Metropolis, one sweep is $M = N^2$ single–spin proposals; for Wolff, we accumulate clusters until the cumulative number of flipped spins reaches $\geq M$, and then record a value. Thus the sampling interval is one *sweep*, and all time–series quantities below are measured in *sweeps*.

*Autocorrelation and IACT.* Following standard MCMC efficiency analysis (e.g., [10, 12]), let $\{H_t\}_{t=1}^n$ be the post–burn-in energy series. Define the sample mean $\bar{H} := \frac{1}{n}\sum_{t=1}^{n} H_t$ and the centered autocovariances

$$\Gamma(k) := \frac{1}{n}\sum_{t=1}^{n-k}(H_t - \bar{H})(H_{t+k} - \bar{H}), \qquad \Gamma(0) := \frac{1}{n}\sum_{t=1}^{n}(H_t - \bar{H})^2.$$

The lag-$k$ autocorrelation is $\rho(k) = \Gamma(k)/\Gamma(0)$. Strong autocorrelation means consecutive samples carry redundant information; the *integrated autocorrelation time* (IACT) therefore quantifies how many sweeps are required to produce one effectively independent draw:

$$\tau_{\text{int}} = 1 + 2\sum_{k=1}^{\infty}\rho(k),$$

estimated via the initial-positive-sequence (IPS) or a window cutoff $K$:

$$\widehat{\tau}_{\text{int}} = 1 + 2\sum_{k=1}^{K}\widehat{\rho}(k).$$

*Effective sample size (ESS).* Autocorrelation inflates the variance of the sample mean:

$$\text{Var}(\bar{H}) \;\approx\; \frac{\sigma_H^2}{n}\,2\tau_{\text{int}}, \qquad \sigma_H^2 := \Gamma(0).$$

Matching this to the variance of a mean based on $n_{\text{eff}}$ i.i.d. samples leads to

$$\boxed{\text{ESS} := \frac{n}{\tau_{\text{int}}}}.$$

In words, among $n$ recorded sweeps only about $n/\tau_{\text{int}}$ contribute independent information, because $\tau_{\text{int}}$ is the number of sweeps needed per effectively independent sample.

*Cost normalization.* To compare algorithms independently of wall–clock time, we count *spin flips* as work:

$$\text{work} = \begin{cases} \text{number of attempted single–spin flips,} & \text{Metropolis,} \\ \text{total spins flipped across all clusters,} & \text{Wolff.} \end{cases}$$

Let $\overline{w} := \text{work}/n$ be the average work per recorded sweep. Then

$$\text{Cost/ESS} = \frac{\text{work}}{\text{ESS}} = \overline{w}\,\tau_{\text{int}}.$$

Hence Cost/ESS measures the number of spin flips needed to obtain one effectively independent sample; smaller is better.

Under our cost normalization (equalizing total spin flips per sweep), we expect the Wolff (cluster) algorithm to yield a smaller $\widehat{\tau}_{\text{int}}$—and hence a lower Cost/ESS—than Metropolis in the critical regime.

## 6. Result and Comparison Analysis.

**6.1. Experiment 1: Validation by Thermodynamics Curves.** On square lattices ($N = 32, 64, 128$), we sweep a temperature grid around the critical point and estimate the per–site internal energy $u(T)$ and specific heat $c(T)$ for both Metropolis and Wolff, see Figure 1 and 2. Both methods reproduce the expected $q = 3$ Potts behavior: a rapid change in $u(T)$ and a pronounced $c(T)$ peak near the theoretical $T_c = 1/\ln(1 + \sqrt{q})$ (vertical dashed line in the figures), and the inferred transition location matches theory across all $N$.

Under equalized work, Wolff generally produces smoother $u(T)$ and $c(T)$ with smaller fluctuations, suggesting better decorrelation per sweep than single–spin Metropolis. A consistent difference appears for Metropolis just *below* $T_c$: $u(T)$ tends to lag the ordered branch and the $c(T)$ peak is broader/noisier—an effect of critical slowing down and domain pinning. Even with a higher budget ($N = 64$, $W_{\text{Metro}}$=1000 vs. $W_{\text{Wolff}}$=500, Figure 1c and 2c), this noise is reduced but not eliminated, whereas Wolff already yields comparatively crisp peaks at fewer sweeps.

Overall, we conclude that:

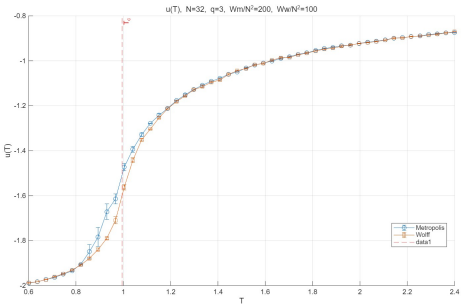
1. Both algorithms are validated by successfully recovering the phase transition, producing $u(T)$ and $c(T)$ curves whose critical behavior aligns with known theoretical predictions.
2. The Wolff (cluster) dynamic demonstrably mitigates the critical slowing down that severely affects single-spin Metropolis updates near $T_c$.
3. A Wolff sweep typically contributes more independent information per unit of flipping work. Consequently, in these settings, Wolff attains stable and smoother thermodynamic curves with less overall Monte Carlo effort.

**6.2. Experiment 2: Comparison of Efficiency Metrics Near Critical.** To quantify performance at the phase transition, we evaluate both algorithms directly at the theoretical critical temperature $T_c = 1/\ln(1 + \sqrt{q})$. We conduct two main comparisons: (i) fixed $q = 3$ with varying system size $N \in \{32, 64, 128\}$, and (ii) fixed size $N = 32$ with varying state number $q \in \{3, 5, 10, 20\}$. Each configuration uses $B = 1000$ burn-in sweeps followed by $n = 10000$ measurement sweeps; one sweep equals $M = N^2$ spin-flip *attempts* for Metropolis and an accumulation of $M$ *actual* flips for Wolff (Section 5). From the post–burn-in energy series $\{H_t\}_{t=1}^{n}$ we estimate the integrated autocorrelation time $\widehat{\tau}_{\text{int}}$ via the IPS method and report the effective sample size $\text{ESS} = n/\widehat{\tau}_{\text{int}}$ and the work–normalized efficiency Cost/ESS = work/ESS.

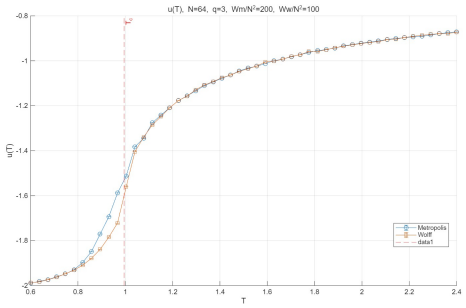
Table 1 summarizes results for $N$-scaling at $q = 3$. Table 2 summarizes results for $q$-scaling at $N = 32$.

Table 1: Dynamics at $T = T_c$ for the $q = 3$ Potts model (varying $N$). Averages (mean ± s.e.) over three replicates; flips are total spin flips.

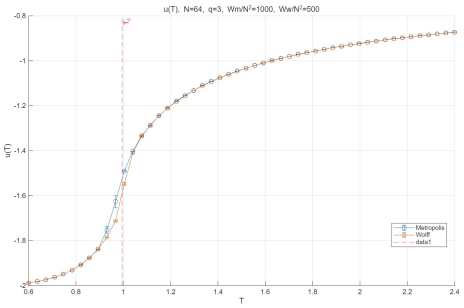
| Setting ($q = 3$) | IACT (sweeps) (↓) | ESS (↑) | Flips | Cost/ESS (↓) |
|---|---|---|---|---|
| $N = 32$ Metropolis | $785.7 \pm 104.6$ | $13 \pm 2$ | $(1.1 \times 10^7) \pm 0$ | $(8.9\times10^5) \pm (1.2\times10^5)$ |
| $N = 32$ Wolff | $12.4 \pm 1.7$ | $837 \pm 102$ | $(1.1 \times 10^7) \pm 47$ | $(1.4\times10^4) \pm (1.9\times10^3)$ |
| $N = 64$ Metropolis | $1439.4 \pm 282.2$ | $7 \pm 1$ | $(4.5 \times 10^7) \pm 0$ | $(6.5\times10^6) \pm (1.3\times10^6)$ |
| $N = 64$ Wolff | $15.5 \pm 1.3$ | $652 \pm 49$ | $(4.5 \times 10^7) \pm 6.1\times10^2$ | $(7.0\times10^4) \pm (5.7\times10^3)$ |
| $N = 128$ Metropolis | 1661.9 | 6 | $\approx 1.8 \times 10^8$ | $3.0 \times 10^7$ |
| $N = 128$ Wolff | 19.0 | 527 | $\approx 1.8 \times 10^8$ | $3.4 \times 10^5$ |



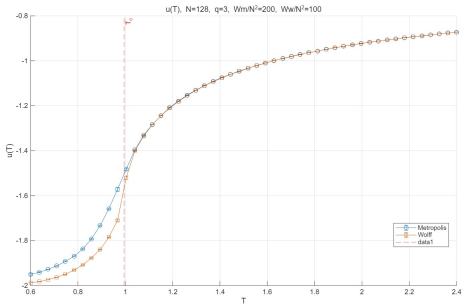

(a) $N = 32$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.




(b) $N = 64$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.




(c) $N = 64$, $W_{\text{Metro}} = 1000$, $W_{\text{Wolff}} = 500$.



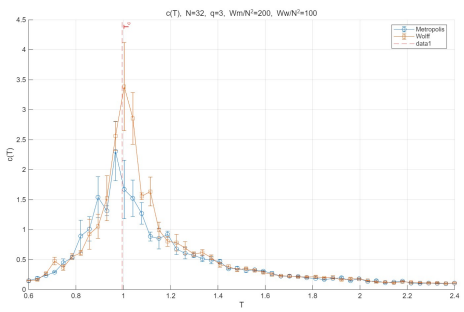

(d) $N = 128$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.

Fig. 1: Per-site internal energy $u(T)$ for Metropolis vs. Wolff under equalized work budgets ($W_{\text{Metro}}$ and $W_{\text{Wolff}}$ are sweeps per temperature as indicated). A vertical line at $T_c = 1/\ln(1+\sqrt{3})$ is included in each panel (in the source figures).

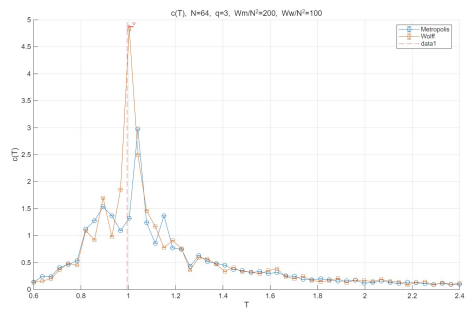
*Analysis of Scaling.* We analyze the impact of both system size $N$ and state number $q$ on algorithm efficiency.

*Scaling with System Size N (Table 1).* At fixed $q = 3$ (a second-order transition), the performance of the two algorithms diverges significantly as $N$ increases. The single-spin Metropolis algorithm exhibits severe critical slowing down: its integrated autocorrelation time ($\widehat{\tau}_{\text{int}}$) grows rapidly with system size $N$, increasing from $\sim 786$ to $\sim 1662$. In sharp contrast, the Wolff update's $\widehat{\tau}_{\text{int}}$ remains remarkably stable and small, consistently on the order of 10–20 sweeps, across all tested lattice sizes.

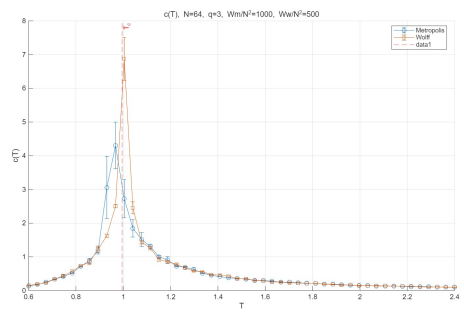
This fundamental difference in decorrelation speed means that for a fixed run length, Wolff yields an Effective Sample Size (ESS) that is two to three orders of magnitude larger. Consequently, the normalized Cost/ESS (spin flips per independent sample) is dramatically lower for Wolff. For instance, at $N = 64$, the costs were $\sim 7.0 \times 10^4$ (Wolff) versus $\sim 6.5 \times 10^6$ (Metropolis). These findings clearly demonstrate that Wolff's collective cluster-flip dynamic is essential for mitigating critical slowing down at the phase transition.



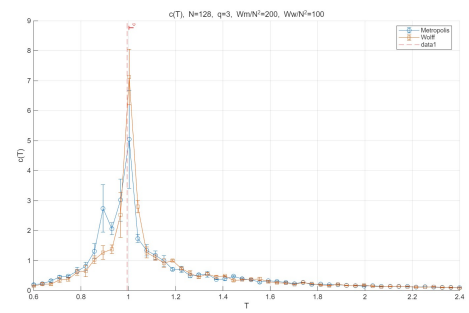

(a) $N = 32$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.




(b) $N = 64$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.




(c) $N = 64$, $W_{\text{Metro}} = 1000$, $W_{\text{Wolff}} = 500$.




(d) $N = 128$, $W_{\text{Metro}} = 200$, $W_{\text{Wolff}} = 100$.

Fig. 2: Per-site specific heat $c(T)$ for Metropolis vs. Wolff under equalized work budgets (sweeps per temperature as indicated). The peak near $T_c = 1/\ln(1+\sqrt{3})$ becomes sharper and smoother for the cluster update, particularly at $N = 64$ high-budget runs (c).

Table 2: Dynamics at $T = T_c$ for fixed $N = 32$ (varying $q$). Averages (mean $\pm$ s.e.) over three replicates. Flips are $\approx 1.1 \times 10^7$ for all runs.

| Setting ($N = 32$) | IACT (sweeps) (↓) | ESS (↑) | Cost/ESS (↓) |
|---|---|---|---|
| $q = 3$ Metropolis | $785.7 \pm 104.6$ | $13 \pm 2$ | $(8.9\times10^5) \pm (1.2\times10^5)$ |
| $q = 3$ Wolff | $12.4 \pm 1.7$ | $837 \pm 102$ | $(1.4\times10^4) \pm (1.9\times10^3)$ |
| $q = 5$ Metropolis | $726.3 \pm 149.5$ | $15 \pm 3$ | $(8.2\times10^5) \pm (1.7\times10^5)$ |
| $q = 5$ Wolff | $48.9 \pm 5.6$ | $209 \pm 22$ | $(5.5\times10^4) \pm (6.3\times10^3)$ |
| $q = 10$ Metropolis | $1379.2 \pm 814.2$ | $14 \pm 6$ | $(1.6\times10^6) \pm (9.2\times10^5)$ |
| $q = 10$ Wolff | $172.3 \pm 67.5$ | $75 \pm 23$ | $(1.9\times10^5) \pm (7.6\times10^4)$ |
| $q = 20$ Metropolis | $178.2 \pm 16.5$ | $57 \pm 5$ | $(2.0\times10^5) \pm (1.9\times10^4)$ |
| $q = 20$ Wolff | $47.8 \pm 6.3$ | $217 \pm 30$ | $(5.4\times10^4) \pm (7.1\times10^3)$ |

*Scaling with State Number q (Table 2).* At fixed $N = 32$, the Wolff algorithm consistently and significantly outperforms Metropolis for all $q$ values tested. The Cost/ESS for Wolff is always at least one order of magnitude lower than for Metropolis (e.g., for $q = 5$, $5.5 \times 10^4$ vs $8.2 \times 10^5$).

The behavior as $q$ increases is complex, as the nature of the phase transition changes from second-order ($q \leq 4$) to first-order ($q > 4$). For Wolff, the efficiency is optimal at $q = 3$ (IACT $\sim 12.4$) and worsens as $q$ increases to 5 and 10 (IACT $\sim 172.3$). This suggests that as the first-order transition becomes stronger (larger $q$), single clusters are less effective at decorrelating the state, which may involve tunneling between two distinct ordered phases. Interestingly, Metropolis shows very poor performance for $q = 3, 5, 10$ but sees a significant improvement in IACT at $q = 20$. Regardless of this complexity, the central conclusion holds: Wolff provides a substantial and robust efficiency gain over Metropolis across all tested $q$ values.

**Acknowledgments and Notes.** All algorithms were re-implemented directly from their mathematical descriptions (no external code bases), and all comparisons avoid wall–clock timing; we normalize work by spin operations and per–sweep sampling throughout.

*Scope and limitations.* Our Wolff implementation uses the standard single–cluster update at $h = 0$; extensions to $h \neq 0$ (e.g., ghost–spin) are outside the present scope. We focus on energy–based metrics; other observables are left for future work.

*Reproducibility & Availability.* Experiments were run in MATLAB R2025b (Linux, 64-bit). Random seeds are fixed per replicate; reported error bars are standard errors across independent runs. Scripts to reproduce all figures and tables, including MATLAB codes and LaTeX source files, are available. **A zip file of MATLAB codes was submitted through Canvas separately.**

**The Use of Large Language Models.** In the preparation of this project, I used ChatGPT as an auxiliary tool in the following limited ways:

1. **Topic exploration and literature orientation:** ChatGPT provided preliminary outline suggestions during topic selection, helped me search for relevant background materials on Cluster Monte Carlo methods, and offered guidance on commonly used parameter choices for benchmarking algorithms.
2. **Language refinement and presentation quality:** I used ChatGPT to polish grammar, improve the fluency of my writing, and generate aesthetically formatted tables and visualizations based on the experimental data I had already produced.
3. **Identification of supplementary numerical observations:** After I obtained the experimental results, ChatGPT pointed out several numerical patterns and phenomena that I had not initially noticed, which helped me refine my discussion.
4. **Code restructuring and readability improvements:** After I hand-wrote the first complete and executable MATLAB implementation, I used ChatGPT to help reorganize the code into a cleaner structure, add more reader-friendly comments, and set up a more rigorous and reproducible experimental comparison framework.

All theoretical derivations, algorithmic designs, experiments, implementation logic, and final interpretations of results are my own work. ChatGPT served only as an assistant for organization, language refinement, code tidying, and high-level guidance.

REFERENCES

[1] G. Winkler. *Image Analysis, Random Fields and Dynamic Monte Carlo Methods*. Springer-Verlag, Berlin and New York, 1995.
[2] R. H. Swendsen and J.-S. Wang. Nonuniversal critical dynamics in Monte Carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987.
[3] U. Wolff. Collective Monte Carlo Updating for Spin Systems. *Physical Review Letters*, 62(4):361–364, 1989.
[4] D. Higdon. Auxiliary Variable Methods for Markov Chain Monte Carlo with Applications. *Journal of the American Statistical Association*, 93(442):585–595, 1998.
[5] C. M. Fortuin and P. W. Kasteleyn. On the Random-Cluster Model: I. Introduction and Relation to Other Models. *Physica*, 57(4):536–564, 1972.
[6] R. G. Edwards and A. D. Sokal. Generalization of the Fortuin–Kasteleyn Random-Cluster Model and Monte Carlo Algorithm. *Physical Review D*, 38(6):2009–2012, 1988.
[7] D. A. Levin, Y. Peres, and E. L. Wilmer. *Markov Chains and Mixing Times*, 2nd ed. American Mathematical Society, 2017.
[8] D. Randall and P. Tetali. Analyzing Glauber dynamics by comparison of Markov chains. *Journal of Mathematical Physics*, 41(3):1598–1615, 2000.
[9] M. Ullrich. Comparison of Swendsen–Wang and single-bond dynamics in the random-cluster model. *Random Structures & Algorithms*, 42(4):520–535, 2013.
[10] C. J. Geyer. Practical Markov Chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992.
[11] A. D. Sokal. Monte Carlo Methods in Statistical Mechanics: Foundations and New Algorithms. In *Functional Integration: Basics and Applications*, 131–192, 1997.
[12] C. P. Robert and G. Casella. *Monte Carlo Statistical Methods*, 2nd ed. Springer, 2004.
[13] R. J. Glauber. Time-Dependent Statistics of the Ising Model. *Journal of Mathematical Physics*, 4(2):294–307, 1963.
[14] R. J. Baxter. *Exactly Solved Models in Statistical Mechanics*. Academic Press, 1982.
[15] G. R. Grimmett. *The Random-Cluster Model*. Springer, 2006.

**Appendix A. Implementation.** The following subsections show the exact implementation of our experiments. A zip file containing MATLAB codes, along with a README file, was submitted through Canvas separately.

**A.1. Validation by Thermodynamics Curves.** MATLAB codes for Experiment 1:

```
clear; clc; rng(1);
%% ---------- Experiment Parameters (Paired k-th items) -----------
N_list = [16, 32]; % List of lattice sizes (N)
q_list = [3, 3]; % List of q values (must align with N_list or be scalar)
W_perT_metro_list = [200, 200]; % Metropolis: target sweeps per temperature (1 sweep = N^2
    flips)
W_perT_wolff_list = [100, 100]; % Wolff: target sweeps per temperature
repeats = 3; % Number of repeats for each parameter set
% Temperature grid and burn-in config (Annealed warm-start: from high T to low T)
T_values = linspace(2.4, 0.6, 50);
scan_burn_sweeps = 20; % Number of sweeps to discard at each temperature point
% Align parameter list lengths: allow some to be scalar, others to have length L
L = max([numel(N_list), numel(q_list), numel(W_perT_metro_list), numel(W_perT_wolff_list)])
    ;
%% ----------------------- Main Loop (Paired k-th items) -----------------------
run_id = 0;
for k = 1:L
  N = pick_k(N_list, k);
  q = pick_k(q_list, k);
  W_perT_metro = pick_k(W_perT_metro_list, k);
  W_perT_wolff = pick_k(W_perT_wolff_list, k);
```

```
M = N*N;
Tc = 1/log(1+sqrt(q)); % kB=J=1
% Statistics containers
uM_all = zeros(repeats, numel(T_values));
cM_all = zeros(repeats, numel(T_values));
uW_all = zeros(repeats, numel(T_values));
cW_all = zeros(repeats, numel(T_values));
for rep = 1:repeats
  run_id = run_id + 1;
  fprintf('Run %d | Scan: N=%d, q=%d, Wm/N^2=%d, Ww/N^2=%d, rep=%d\n', ...
          run_id, N, q, W_perT_metro, W_perT_wolff, rep);
  % Annealed warm-start: from high T to low T; functions must return sigma_* to preserve it for the next T point
  sigma_m = randi([1 q], N, N);
  sigma_w = randi([1 q], N, N);
  u_m = zeros(1, numel(T_values)); c_m = u_m;
  u_w = zeros(1, numel(T_values)); c_w = u_w;
  for t_index = 1:numel(T_values)
    T = T_values(t_index);
    beta = 1/T; %#ok<NASGU> % Used for formula explanation, value is calculated in sub-functions
    % --- Metropolis: Collect samples for equivalent work ---
    W_target_m = W_perT_metro * M;
    [Hs_m, ~, sigma_m] = collect_sweeps_equal_work_metro(sigma_m, T, W_target_m, q);
    H_use = drop_burn(Hs_m, scan_burn_sweeps);
    u_m(t_index) = mean(H_use)/M;
    c_m(t_index) = ( (1/T)^2 * var(H_use) )/M;
    % --- Wolff: Collect samples for equivalent work ---
    W_target_w = W_perT_wolff * M;
    [Hs_w, ~, sigma_w] = collect_sweeps_equal_work_wolff(sigma_w, T, W_target_w, q);
    H_use = drop_burn(Hs_w, scan_burn_sweeps);
    u_w(t_index) = mean(H_use)/M;
    c_w(t_index) = ( (1/T)^2 * var(H_use) )/M;
  end
  uM_all(rep,:) = u_m; cM_all(rep,:) = c_m;
  uW_all(rep,:) = u_w; cW_all(rep,:) = c_w;
end
% Calculate mean and standard error
mu_uM = mean(uM_all,1); se_uM = std(uM_all,0,1)/sqrt(repeats);
mu_cM = mean(cM_all,1); se_cM = std(cM_all,0,1)/sqrt(repeats);
mu_uW = mean(uW_all,1); se_uW = std(uW_all,0,1)/sqrt(repeats);
mu_cW = mean(cW_all,1); se_cW = std(cW_all,0,1)/sqrt(repeats);
% --- Plot: u(T) ---
figure('Name', sprintf('u(T): N=%d, q=%d, Wm/N^2=%d, Ww/N^2=%d',N,q,W_perT_metro, W_perT_wolff)); hold on;
errorbar(T_values, mu_uM, se_uM, '-o', 'DisplayName','Metropolis');
errorbar(T_values, mu_uW, se_uW, '-s', 'DisplayName','Wolff');
xline(Tc,'r--','T_c'); xlabel('T'); ylabel('u(T)');
title(sprintf('u(T), N=%d, q=%d, Wm/N^2=%d, Ww/N^2=%d', N, q, W_perT_metro, W_perT_wolff));
```

```
  legend('Location','best'); grid on;
  % --- Plot: c(T) ---
  figure('Name', sprintf('c(T): N=%d, q=%d, Wm/N^2=%d, Ww/N^2=%d',N,q,W_perT_metro,
      W_perT_wolff)); hold on;
  errorbar(T_values, mu_cM, se_cM, '-o', 'DisplayName','Metropolis');
  errorbar(T_values, mu_cW, se_cW, '-s', 'DisplayName','Wolff');
  xline(Tc,'r--','T_c'); xlabel('T'); ylabel('c(T)');
  title(sprintf('c(T), N=%d, q=%d, Wm/N^2=%d, Ww/N^2=%d', N, q, W_perT_metro, W_perT_wolff)
      );
  legend('Location','best'); grid on;
end
%% ======================== Local Helper Functions =========================
function v = pick_k(lst, k)
  if numel(lst) == 1
    v = lst(1);
  else
    v = lst(k);
  end
end
function H_use = drop_burn(Hs, burn)
  if numel(Hs)>burn, H_use = Hs(burn+1:end);
  else, H_use = Hs;
  end
end
function [H_sweeps, flips, sigma] = collect_sweeps_equal_work_metro(sigma, T, W_target, q)
  % Single-spin proposal + Metropolis acceptance; 1 sweep per N^2 flips
  [N,~] = size(sigma); beta = 1/T; M = N*N;
  H_cur = calculateH(sigma, 1, 0);
  flips = 0; in_sweep = 0; H_sweeps = [];
  while flips < W_target
    i = randi([1 N]); j = randi([1 N]);
    newv = mod(sigma(i,j)+randi([1 q-1])-1, q)+1;
    dH = calculateDeltaH(newv, sigma, i, j, 1, 0);
    if dH < 0 || rand < exp(-beta*dH)
      sigma(i,j) = newv;
      H_cur = H_cur + dH;
    end
    flips = flips + 1;
    in_sweep = in_sweep + 1;
    if in_sweep >= M
      H_sweeps(end+1,1) = H_cur; %#ok<AGROW>
      in_sweep = 0;
    end
  end
end
function [H_sweeps, flips, sigma] = collect_sweeps_equal_work_wolff(sigma, T, W_target, q)
  % Cluster growth + immediate recolor; accumulate cluster size as flips; 1 sweep per >= N
      ^2 accumulated flips
  [N,~] = size(sigma); beta = 1/T; M=N*N; p = 1 - exp(-beta*1);
  H_cur = calculateH(sigma, 1, 0);
```

```
  flips = 0; carry = 0; H_sweeps = [];
  while flips < W_target
    x = randi(N); y = randi(N);
    old = sigma(x,y);
    new = mod(old + randi([1 q-1]) - 1, q) + 1;
    cluster = [x, y]; sigma(x,y) = new; tail = 1;
    while ~isempty(cluster)
      [cx, cy] = deal(cluster(end,1), cluster(end,2));
      cluster(end,:) = [];
      neighbors = [mod(cx-2,N)+1, cy;
                   mod(cx, N)+1, cy;
                   cx, mod(cy-2,N)+1;
                   cx, mod(cy, N)+1];
      for ii=1:4
        xn = neighbors(ii,1); yn = neighbors(ii,2);
        if sigma(xn,yn)==old && rand<p
          sigma(xn,yn)=new; cluster=[cluster; xn, yn]; %#ok<AGROW>
          tail = tail + 1;
        end
      end
    end
    H_cur = calculateH(sigma, 1, 0);
    flips = flips + tail; carry = carry + tail;
    while carry >= M
      H_sweeps(end+1,1) = H_cur; %#ok<AGROW>
      carry = carry - M;
    end
  end
end
```

**A.2. Comparison of Efficiency Metrics Near Critical.** MATLAB codes for Experiment 2:

```
clear; clc; rng(2);
%% --------- Experiment Parameters (Paired k-th items) -----------
N_list = [16, 32]; % Can be scalar or list of equal length
q_list = [3, 3]; % Can be scalar or list of equal length
repeats = 2;
% Sampling settings (shared by both algorithms; add new parameters if separate settings are
    needed)
burn_sweeps = 100;
keep_sweeps = 2000;
maxlag_dyn = floor(keep_sweeps/5);
% Align list lengths: allow some to be scalar, others to have length L
L = max([numel(N_list), numel(q_list)]);
%% ----------------------- Main Loop (Paired k-th items) -----------------------
run_id = 0;
for k = 1:L
  N = pick_k(N_list, k);
  q = pick_k(q_list, k);
```

```
  M = N*N;
  Tc = 1/log(1+sqrt(q)); % J=kB=1
  fprintf('\n=== Dynamics @Tc: N=%d, q=%d (Tc=%.6f) ===\n', N, q, Tc);
  % stats columns: [tau, ESS, flips, Cost/ESS]
  statsM = zeros(repeats, 4);
  statsW = zeros(repeats, 4);
  for rep = 1:repeats
    run_id = run_id + 1;
    % --- Metropolis ---
    sigma0 = randi([1 q], N, N);
    [H_m, flips_m] = metropolis_sweeps_series(sigma0, Tc, burn_sweeps, keep_sweeps, 1, 0, q);
    [tau_m, ~, K_m] = iact_ips(H_m, maxlag_dyn);
    ESS_m = max(numel(H_m)/max(tau_m, eps), 0); % Protect against division by zero
    cost_m = flips_m / max(ESS_m, eps);
    statsM(rep,:) = [tau_m, ESS_m, flips_m, cost_m];
    % --- Wolff ---
    sigma0 = randi([1 q], N, N);
    [H_w, flips_w] = wolff_sweeps_series(sigma0, Tc, burn_sweeps, keep_sweeps, 1, 0, q);
    [tau_w, ~, K_w] = iact_ips(H_w, maxlag_dyn);
    ESS_w = max(numel(H_w)/max(tau_w, eps), 0);
    cost_w = flips_w / max(ESS_w, eps);
    statsW(rep,:) = [tau_w, ESS_w, flips_w, cost_w];
    fprintf('Run %d | rep %d | Metro: IACT=%.1f (K~%d), ESS=%.0f, Cost/ESS=%.2g | Wolff: IACT=%.1f (K~%d), ESS=%.0f, Cost/ESS=%.2g\n',...
      run_id, rep, tau_m, K_m, ESS_m, cost_m, tau_w, K_w, ESS_w, cost_w);
  end
  muM = mean(statsM,1); seM = std(statsM,0,1)/sqrt(repeats);
  muW = mean(statsW,1); seW = std(statsW,0,1)/sqrt(repeats);
  fprintf('Avg Metro: IACT=%.1f%.1f, ESS=%.0f%.0f, flips=%.2g%.2g, Cost/ESS=%.2g%.2g\n',...
    muM(1), seM(1), muM(2), seM(2), muM(3), seM(3), muM(4), seM(4));
  fprintf('Avg Wolff: IACT=%.1f%.1f, ESS=%.0f%.0f, flips=%.2g%.2g, Cost/ESS=%.2g%.2g\n',...
    muW(1), seW(1), muW(2), seW(2), muW(3), seW(3), muW(4), seW(4));
end
%% ======================== Local Helper Functions =====================
function v = pick_k(lst, k)
  if numel(lst) == 1, v = lst(1); else, v = lst(k); end
end
function [H_sweeps, flips_total] = metropolis_sweeps_series(sigma, T, burn_sweeps, keep_sweeps, J, h, q)
  [N,~] = size(sigma); beta = 1/T; M=N*N;
  H_cur = calculateH(sigma, J, h);
  total_sweeps = burn_sweeps + keep_sweeps;
  flips_total = 0; rec = 0; H_sweeps = zeros(keep_sweeps,1); sweeps_done = 0;
  while sweeps_done < total_sweeps
    for t = 1:M
      i=randi([1 N]); j=randi([1 N]);
      newv = mod(sigma(i,j)+randi([1 q-1])-1, q)+1;
      dH = calculateDeltaH(newv, sigma, i, j, J, h);
      if dH < 0 || rand < exp(-beta*dH)
```

```
      sigma(i,j) = newv; H_cur = H_cur + dH;
    end
    flips_total = flips_total + 1;
  end
  sweeps_done = sweeps_done + 1;
  if sweeps_done > burn_sweeps
    rec = rec + 1; H_sweeps(rec) = H_cur;
    if rec == keep_sweeps, return; end
  end
 end
end
function [H_sweeps, flips_total] = wolff_sweeps_series(sigma, T, burn_sweeps, keep_sweeps,
    J, h, q)
 [N,~] = size(sigma); beta = 1/T; M=N*N;
 H_cur = calculateH(sigma, J, h);
 p = 1 - exp(-beta*J);
 flips_total = 0; H_sweeps = zeros(keep_sweeps,1);
 sweeps_done = 0; carry = 0; rec = 0;
 while sweeps_done < burn_sweeps + keep_sweeps
  x=randi(N); y=randi(N);
  old = sigma(x,y);
  new = mod(old + randi([1 q-1]) - 1, q) + 1;
  cluster=[x,y]; sigma(x,y)=new; tail=1;
  while ~isempty(cluster)
    [cx,cy] = deal(cluster(end,1), cluster(end,2)); cluster(end,:)=[];
    neighbors = [mod(cx-2,N)+1, cy;
                 mod(cx, N)+1, cy;
                 cx, mod(cy-2,N)+1;
                 cx, mod(cy, N)+1];
    for ii=1:4
      xn=neighbors(ii,1); yn=neighbors(ii,2);
      if sigma(xn,yn)==old && rand < p
        sigma(xn,yn)=new; cluster=[cluster; xn, yn]; %#ok<AGROW>
        tail = tail + 1;
      end
    end
  end
  H_cur = calculateH(sigma, J, h);
  flips_total = flips_total + tail; carry = carry + tail;
  while carry >= M
    sweeps_done = sweeps_done + 1;
    if sweeps_done > burn_sweeps
      rec = rec + 1; H_sweeps(rec) = H_cur;
      if rec == keep_sweeps, return; end
    end
    carry = carry - M;
  end
 end
end
function [tau_int, rho, K] = iact_ips(x, maxlag)
```

```
  x = x(:); n = numel(x);
  x = x - mean(x);
  c0 = sum(x.^2) / n;
  rho = zeros(maxlag,1);
  for k = 1:maxlag
    rho(k) = sum( x(1:n-k).*x(1+k:n) ) / ((n-k)*c0);
    if rho(k) <= 0
      rho = rho(1:k-1); break;
    end
  end
  if isempty(rho), rho = 0; end
  K = numel(rho);
  tau_int = 1 + 2*sum(rho);
end
```

**A.3. Other Helper Functions.** Helper functions that calculate the (change of) Hamiltonian:

```
% Function to calculate the Hamiltonian
function result = calculateH(sigma, J, h)
% Get the size of the lattice
[N, M] = size(sigma);

% Initialize the Hamiltonian
H = zeros(N, M);

% Calculate the Hamiltonian
for i = 1:N
    for j = 1:M
        H(i, j) = -J * ((sigma(i, j) == sigma(mod(i-2, N)+1, j)) + (sigma(i, j) == sigma(
            mod(i, N)+1, j)) + (sigma(i, j) == sigma(i, mod(j-2, M)+1)) + (sigma(i, j) ==
            sigma(i, mod(j, M)+1)));
    end
end
result = sum(H(:))/2;
for i = 1:N
    for j = 1:M
        result = result-H(i,j)*h;
    end
end
end

% Calculate the change in Hamiltonian
function deltaH = calculateDeltaH(value, sigma, i, j, J, h)
% Get the size of the lattice
[N, M] = size(sigma);

% Calculate the original Hamiltonian
H0 = -J * ((sigma(i, j) == sigma(mod(i-2, N)+1, j)) + (sigma(i, j) == sigma(mod(i, N)+1, j)
    ) + (sigma(i, j) == sigma(i, mod(j-2, M)+1)) + (sigma(i, j) == sigma(i, mod(j, M)+1)))
    ;
```

```
H1 = -J * ((value == sigma(mod(i-2, N)+1, j)) + (value == sigma(mod(i, N)+1, j)) + (value
     == sigma(i, mod(j-2, M)+1)) + (value == sigma(i, mod(j, M)+1)));
H0 = H0 - h*sigma(i, j);
H1 = H1 - h*value;

% Calculate the change in Hamiltonian
deltaH = (H1-H0);
end
```